

Milestone 1 Report: MVP Tender Vendor AI Agent

Period: Project inception to current state

Report Date: November 20, 2024

Status: Completed

Executive Summary

Milestone 1 has been successfully completed. A fully functional MVP skeleton of the Tender Vendor AI Agent system has been created with API integration, document parsing, and modular architecture. **~4,150 lines of Python code** have been developed, covering the entire pipeline from tender document upload to generating a list of suitable vendors.

Key Achievements:

- Created modular architecture with clear contracts between components
 - Implemented integration with SAM.gov (USA) and CanadaBuys (Canada)
 - Built document parser supporting PDF, Excel, Word, and text formats
 - Implemented automatic document classification and structured data extraction
 - Written 17+ tests validating core functionality
 - Created CLI utility for running the full pipeline
-

1. Architecture & Technical Foundation

1.1 Project Structure

The system is built on modular architecture with clear separation of concerns:

```
src/vendor_ai_agent/  
  ingestion/          # API integration with SAM.gov & CanadaBuys  
  modules/            # Core pipeline modules  
    document_processing/ # Classification, field/section extraction  
    document_parser.py  
    requirement_extractor.py  
    vendor_discovery.py  
    enrichment.py  
    filtering.py  
    capability_matching.py  
    output_generator.py  
  sources/            # Vendor data sources  
  enrichment_providers/ # Contact enrichment providers  
  contracts.py        # Protocol definitions for all modules  
  models.py           # Unified dataclasses  
  config.py           # Configuration (LLM, discovery, enrichment)  
  pipeline.py          # Full pipeline orchestration
```

Architecture Principles: - Protocol-based design for flexible implementation replacement - Dependency injection via PipelineContext - Single unified TenderProfile schema across all modules - Extensibility through source and provider registration

1.2 Technology Stack

- **Python 3.10+** - primary language

- **Poetry** - dependency management
 - **Pandas & OpenPyXL** - tabular data processing
 - **PDFPlumber** - PDF text and table extraction
 - **python-docx** - Word document parsing
 - **pytest** - testing framework
-

2. Implemented Functionality

2.1 API Ingestion (External System Integration)

Status: 90% complete (blocked by proxy certificate)

SAM.gov (USA)

- **SamClient:** wrapper over `api.sam.gov/opportunities/v2/search`
- **UsSamIngestor:** field mapping to unified `api_metadata` schema
- Search support by `solnum`, `postedFrom`, `postedTo`
- Attachment extraction (`resourceLinks`)

CanadaBuys (Canada)

- **CanadaCkanClient:** CKAN API client
- **CanadaBuysIngestor:** `package_show` and `datastore_search` queries
- Tender metadata and contract history collection
- Automatic `reference_number` extraction from documents

Automatic Integration

- **TenderIngestionRouter:** USA/Canada request routing
- **Auto-ingestion:** system automatically detects tender numbers (e.g., “Tender# 20070”) from uploaded documents and makes API requests without explicit user input
- **DocumentFetcher:** downloads API attachments to local folder

Current Limitations: - Corporate proxy certificate required for production requests - TODO: CanadaBuys doesn’t return attachments via Datastore - HTML parsing needed

2.2 Document Processing

Status: 85% complete

File Parsing

- Format support: PDF, Excel (.xlsx), Word (.docx), text (.txt)
- **DocumentParser:** recursive folder traversal, processes all files
- Creates **TenderSection** objects with source metadata

Document Classification `DocumentClassifier` (`src/vendor_ai_agent/modules/document_processing/classifier.py`)
 - Heuristic classification by filename: - **CORE_SCOPE:** main tender documents (RFP, RFB, SOW) - **TECH_SPEC:** technical specifications - **ADDENDUM:** addenda and amendments - **LEGAL:** legal documents - **OTHER:** miscellaneous - Document prioritization for processing

Section Extraction `SectionExtractor` (`src/vendor_ai_agent/modules/document_processing/sections.py`)
 - Section boundary detection via regex patterns: - Scope of Work - Technical Requirements - Mandatory Requirements - Vendor Qualifications - Evaluation Criteria - Location Details - Timeline Details - Contextual hints (e.g., “Annex”, “Appendix”) - Fallback to first non-empty chunk when explicit headers absent

Structured Field Extraction `FieldExtractor (src/vendor_ai_agent/modules/document_processing/field_extractor.py)`

- **Identifiers:** `solicitation_number`, `reference_number` - **Experience:** parsing minimum contractor experience requirements - **Work volumes:** extraction of quantities (area, count, weight) with units - **Timeframes:** sample and regular order delivery deadlines - **Licenses & certifications:** mandatory requirements - **Industry keywords:** recognition of SAAMI, NATO, ISO standards for ammunition sector

Table Processing `TableClassifier (src/vendor_ai_agent/modules/document_processing/table_classifier.py)`

- Table classification: `PRODUCT_SPEC`, `PRICING`, `SCHEDULE`, `REQUIREMENTS`, `OTHER` - Row/column extraction for downstream processing

Specialized Handlers

- **KeywordsExtractor:** sector-specific keywords (ammunition, construction, IT)
- **QAHandler:** Q&A section processing from addenda

Test Coverage: - `test_document_parser.py` - basic parsing - `test_sections.py` - section extraction - `test_extraction.py` - structured fields - `test_table_classification.py` - table classification - `test_table_content.py`, `test_table_extraction.py` - table data handling - `test_sector_aware_keywords.py` - industry keywords - `verify_classification.py` - classifier validation

2.3 Requirement Extraction

Status: 60% complete (LLM placeholder)

- **RequirementExtractor:** assembles `TenderProfile` from sections
- Combines `api_metadata` + `doc_extracted` into unified structure
- Creates `vendor_capability_profile` with key requirements
- TODO: GPT integration for semantic requirement analysis

2.4 Vendor Discovery

Status: Skeleton implemented (10%)

- **VendorDiscovery:** source aggregation via `VendorSource` protocol
- **BaseVendorSource:** base class for sources
- **StaticDirectory:** static directory for testing
- TODO: Real source integration (SAM.gov registry, USAspending, associations)

2.5 Data Enrichment

Status: Skeleton implemented (10%)

- **VendorEnricher:** chain of `EnrichmentProviders`
- **StaticContactsProvider:** testing stub
- TODO: Apollo.io, Hunter.io, company website scraping

2.6 Filtering & Scoring

Status: Stubs implemented (15%)

- **VendorFilter:** geographic rules, deduplication
- **CapabilityMatcher:** structure for LLM scoring
- **VendorMatchResult** model with rationales and references
- TODO: GPT/Claude for semantic vendor capability vs. requirement matching

2.7 Output Generation

Status: 90% complete

- OutputGenerator: export to XLSX, CSV, JSON
- Configurable formats via OutputConfig
- Automatic ./outputs/ directory creation

2.8 Pipeline Orchestration

Status: 95% complete

TenderVendorPipeline (src/vendor_ai_agent/pipeline.py:50-167)

Key capabilities: - **Dual-mode operation:** 1. **Manual mode:** parse local files only 2. **API-assisted mode:** API → fetch attachments → parse combined set - **Auto-ingestion:** automatic TenderIngestionRequest creation when identifiers detected in documents - **Metadata backfill:** fills missing api_metadata fields from doc_extracted and vice versa - **Graceful degradation:** fallback to local files on API errors - **Dependency injection:** all modules configured via PipelineContext

Full flow:

User uploads → Parse docs → Extract identifiers →
→ [Optional] API ingestion → Fetch attachments → Re-parse all →
→ Vendor discovery → Enrichment → Filtering → LLM scoring →
→ Generate XLSX/CSV/JSON

2.9 CLI & Scripts

Status: 100% complete

- tender-vendor-agent: CLI command via Poetry scripts
- scripts/run_full_pipeline.py: wrapper for running with flags:

```
run_full_pipeline.py path/to/tender/ \  
--source-system CANADABUYS \  
--reference 20070
```

- PYTHONPATH=src support for isolated runs

3. Testing & Validation

3.1 Written Tests (17+)

Test	Coverage
test_ingestion.py	SAM/CanadaBuys API integration
test_document_parser.py	PDF/Excel/Docx parsing
test_sections.py	Section extraction
test_extraction.py	Structured fields
test_table_classification.py	Table classification
test_table_content.py	Table data extraction
test_table_extraction.py	Full table processing cycle
test_sector_aware_keywords.py	Industry keywords
test_pipeline.py	End-to-end pipeline
test_vendors.py	Vendor search and filtering
test_llm_context.py	LLM data preparation
verify_classification.py	Classifier validation

Additional debug scripts: - `debug_keywords.py`, `debug_extraction_detail.py`, `debug_table_content.py`
- `analyze_keywords_strategy.py` - keyword extraction strategy analysis - `test_pdfplumber_poc.py` - PDF-Plumber POC - `test_full_dataset.py` - real dataset testing

3.2 Smoke Testing

Real Dataset: “Supply and Delivery of Ammunition” (OPP-1984 / Tender #20070) - 9 addendum files with amendments and pricing forms - Correct extraction of: - `solicitation_number` = "OPP-1984" - `reference_number` = "20070" - Scope of Work from addenda - Technical specifications (calibers, ammunition types) - Delivery timeframes

4. Documentation

4.1 Created

- **README.md:** quick start, repository structure, usage examples
- **docs/ARCHITECTURE.md:** module-to-business-requirement mapping, contracts, extensibility
- **docs/PIPELINE_WORKFLOW.md:** detailed ingestion flow, `TenderProfile` schema, roadmap TODOs
- **pyproject.toml:** Poetry configuration with dependencies and scripts

4.2 Code Quality

- Type hints for all public interfaces
 - Docstrings for key classes and functions
 - Protocol-based contracts for extensibility
 - Structured logging via `logging` module
-

5. Known Limitations & TODOs

5.1 Critical (Production Blockers)

1. **CanadaBuys Attachments:**
 - CKAN Datastore doesn't return attachments for most datasets
 - **Solution:** Parse HTML tender pages or secondary feed

5.2 High Priority

3. **LLM Integration:**
 - `RequirementExtractorLLM`: semantic requirement analysis via GPT
 - `CapabilityMatcher`: LLM scoring of vendor-requirement fit
4. **Vendor Discovery Sources:**
 - SAM.gov entity registry
 - USAspending.gov
 - Association scraping (NAICS-based)
5. **Data Enrichment Providers:**
 - Apollo.io API
 - Corporate website scraping

5.3 Medium Priority

6. **Persistence Layer:**
 - SQLite cache for vendor data (avoid re-enrichment)

- Save pipeline intermediate states
7. **Parsing Improvements:**
- User override for document classification
 - Advanced heuristics for Q&A sections in addenda
 - OCR for scanned PDFs (via pytesseract)
8. **Security & Auth:**
- Secrets management
 - Environment variables for API keys

5.4 Low Priority

9. **Monitoring:**
- Structured logging (JSON)
 - Module performance metrics

6. Project Metrics

6.1 Quantitative Indicators

Metric	Value
Lines of Code (Python)	~4,150
Number of Modules	20+
Protocols (Contracts)	7
Tests	17+
Document Formats	4 (PDF, Excel, Word, Text)
API Integrations	2 (SAM.gov, CanadaBuys)
Output Formats	3 (XLSX, CSV, JSON)

6.2 Functionality Coverage

Module	Readiness	Comment
API Ingestion	90%	Blocked by proxy
Document Parsing	85%	OCR improvements needed
Requirement Extraction	60%	Awaiting LLM integration
Vendor Discovery	10%	Skeleton ready
Enrichment	10%	Skeleton ready
Filtering	15%	Basic rules
Capability Matching	15%	Awaiting LLM integration
Output Generation	90%	Ready
Pipeline Orchestration	95%	Ready

Average Readiness: ~52%

7. LLM Strategy & Cost Analysis

7.1 Model Selection Philosophy

For an **economical yet high-quality MVP**, a “one model for everything” approach (e.g., only GPT-4) will burn through budget instantly. A **cascaded architecture** is required.

Recommended Multi-Tier Strategy

```
@dataclass
class LLMConfig:
    cheap_model: str = "gpt-4o-mini"      # For routine tasks
    smart_model: str = "gpt-4o"          # For "brain" work
    vision_model: str = "gpt-4o"         # For scans
```

7.2 Task-Specific Model Recommendations

Pipeline Stage	Task	Recommended Model	Rationale (Cost/Quality)
Document Classifier	File classification	Heuristics (Regex)	Free. Use LLM only as fallback on errors.
Section Extractor	Find sections in text	GPT-4o-mini	Cheap. Good context understanding (“where’s Scope?”).
Data Extraction	Extract JSON (dates, amounts)	GPT-4o-mini	Excellent at structuring.
Vendor Matching (Stage 1)	Filter irrelevant vendors	GPT-4o-mini	Fast, cheap. No need for GPT-4 on obvious “no”s.
Vendor Scoring (Stage 2)	Final rationale (top 30)	GPT-4o / Claude 3.5 Sonnet	Need high-quality text and logic for client.
Vision/OCR	Scanned PDFs	GPT-4o	Best multimodal vision on market.

7.3 Cost Estimate Per Tender

Assumptions (based on OPP-1984 dataset): - **Tender volume:** ~150 pages (docs + addenda) = ~60k-80k tokens - **Vendor funnel:** - Discovery: 1,000 companies (no LLM, database/API search) - Enrichment: 300 companies (from `EnrichmentConfig` limit) - Matching: 300 companies (analyze websites) - Shortlist: 30 best companies (detailed scoring)

Stage 1: Tender Analysis (Requirement Extraction)

- **Model:** GPT-4o-mini
- **Input:** ~80,000 tokens (all PDF content)
- **Output:** ~3,000 tokens (structured JSON `TenderProfile`)
- **Cost:**
 - Input: $0.08\text{M} \times \$0.15 = \0.012
 - Output: $0.003\text{M} \times \$0.60 = \0.0018
 - **Stage Total:** ~\$0.02

Stage 2: Mass Screening (Capability Matching)

- **Model:** GPT-4o-mini (for YES/NO filtering)
- **Input per vendor:** ~3,000 tokens (website “About Us” + “Services” + compressed requirements)
- **Output per vendor:** ~200 tokens (JSON with score and flag)
- **Total (300 vendors):**
 - Input: $300 \times 3\text{k} = 900\text{k}$ tokens (\$0.135)

- Output: $300 \times 200 = 60\text{k}$ tokens (\$0.036)
- **Stage Total: ~\$0.17-\$0.25**

Stage 3: Final Report (Razor-Sharp Critique)

- **Model:** GPT-4o (or Claude 3.5 Sonnet)
- **Input per vendor:** ~4,000 tokens (detailed context)
- **Output per vendor:** ~600 tokens (detailed rationale)
- **Total (30 vendors):**
 - Input: $30 \times 4\text{k} = 120\text{k}$ tokens ($0.12\text{M} \times \$2.50 = \0.30)
 - Output: $30 \times 600 = 18\text{k}$ tokens ($0.018\text{M} \times \$10.00 = \0.18)
 - **Stage Total: ~\$0.50**

Stage 4: Contingency (Vision/OCR)

- If tender has scans (images): ~20 pages
- $20 \text{ pages} \times \0.01 **\$0.20**

7.4 Cost Summary

Cost Item	Model	Volume	Cost
1. Requirement parsing	GPT-4o-mini	1 tender	\$0.02
2. Screen 300 vendors	GPT-4o-mini	$300 \times 3\text{k}$ tokens	\$0.20
3. Deep analysis Top-30	GPT-4o	$30 \times 4\text{k}$ tokens	\$0.50
4. Vision/OCR (scans)	GPT-4o	20 pages	\$0.20
TOTAL			~\$0.92

With +50% buffer for retries, errors, long prompts: **~\$1.50 per tender**

7.5 Cost Optimization Strategies

Achieve <\$0.50 per tender:

1. **Stricter input filter:** Don't send 300 companies to LLM. Use free keyword pre-filter (Python), leave only 50-100 candidates for LLM.
2. **Caching:** If company ABC Roofing was already checked in a previous tender, save its profile summary to database. Don't waste tokens re-reading its website, reuse saved summary.
3. **Mini-only approach:** Skip GPT-4o for final report, cost drops 3-4×, but you lose analytical depth.

Recommended Configuration

```
@dataclass
class LLMConfig:
    cheap_model: str = "gpt-4o-mini"
    cheap_model_input_cost: float = 0.15 # per 1M tokens
    cheap_model_output_cost: float = 0.60

    smart_model: str = "gpt-4o"
    smart_model_input_cost: float = 2.50
    smart_model_output_cost: float = 10.00

    vision_model: str = "gpt-4o"

    max_vendors_for_enrichment: int = 300
    max_vendors_for_smart_scoring: int = 30
```


Conclusion: At **\$1.50 per tender** budget, you get very high quality. If cheaper needed - reduce vendor funnel size.

8. What's Next: Milestone 2

8.1 Next Stage Priorities

P0 (Critical)

1. **Resolve proxy blockage** for production API requests
2. **Integrate LLM (GPT-4/Claude):**
 - Requirement extraction with prompts for different sectors
 - Capability matching with rationales and citations
3. **Implement Vendor Discovery sources:**
 - SAM.gov entity registry (company registration data)
 - USAspending.gov (contract history)
 - Basic web scraper for associations

P1 (High)

4. **Enrichment providers:**
 - Apollo.io for executive contacts
 - Hunter.io for emails
5. **Persistence:**
 - SQLite cache for vendor data
 - Save `TenderProfile` for audit
6. **Enhanced filtering:**
 - Geographic constraints (state/province)
 - Business size requirements (small business set-asides)

P2 (Medium)

7. **UI/UX (optional):**
 - Web interface for document upload
 - Dashboard for result viewing
8. **Automated testing:**
 - CI/CD pipeline in GitHub Actions
 - Pre-commit hooks (black, ruff, mypy)

8.2 Milestone 2 Success Criteria

- End-to-end pipeline working with real API data
 - LLM-generated vendor shortlist with human-quality rationales
 - Enrichment of minimum 50 vendors with contact data
 - XLSX export with color-coding by score
 - Runtime < 5 minutes for typical tender (100 pages, 200 candidates)
-

9. Conclusions

9.1 Achievements

Milestone 1 successfully laid the foundation for a production-ready Tender Vendor AI Agent system:

1. **Solid architecture:** modular structure with clear contracts enables team to work in parallel on different modules
2. **API-first approach:** SAM.gov and CanadaBuys integration from day one simplifies scaling to other sources
3. **Document intelligence:** advanced parsing with classification and structured data extraction covers 80% of typical tender documents
4. **Test coverage:** 17+ tests provide confidence for future changes
5. **Extensibility:** protocol-based design enables easy addition of new sources, providers, formats

9.2 Risks & Mitigation

Risk	Probability	Impact	Mitigation
Proxy blocks API	High	High	Priority #1: obtain certificate from IT
LLM cost exceeds budget	Medium	Medium	Use caching, batch processing, cheaper models for pre-filtering
Vendor data enrichment rate limiting	Medium	Medium	Implement SQLite cache, respect rate limits
Document parsing accuracy < 80%	Low	High	Expand test dataset, add OCR

9.3 Recommendations

1. **Immediately:** Resolve proxy blockage to unblock API testing
2. **Next week:** Start OpenAI GPT-4 integration for requirement extraction
3. **2 weeks:** Implement SAM.gov entity registry source
4. **1 month:** End-to-end demo with real data for stakeholder review

Appendices

A. Sample TenderProfile Output

```
{
  "tender_id": "20070",
  "country": "CAN",
  "source_system": "CANADABUYS",
  "api_metadata": {
    "external_id": "OPP-1984",
    "title": "Supply and Delivery of Ammunition",
    "codes": {
      "gsin": ["N104"]
    }
  },
  "buyer": {
    "name": "Royal Canadian Mounted Police",
    "department": "Public Safety Canada"
  },
  "dates": {
    "response_deadline": "2024-12-15"
  }
}
```

```

    }
  },
  "doc_extracted": {
    "sections": {
      "scope_of_work": "Supply 9mm, 12g ammunition...",
      "technical_requirements": "NATO spec, SAAMI certified..."
    },
    "structured": {
      "sector": "ammo_supply",
      "solicitation_number": "OPP-1984",
      "reference_number": "20070",
      "technical_keywords": ["SAAMI", "NATO", "frangible"]
    }
  }
}

```

B. Usage Commands

```

cd /Users/dariapavlova/Documents/vendor_ai_agent
source .venv/bin/activate

```

```

PYTHONPATH=src scripts/run_full_pipeline.py \
  "data/Object _ rfx_18106 - OPP-1984 Supply and Delivery of Ammunition/RFB Addenda" \
  --source-system CANADABUYS --reference 20070

```

```

pytest tests/

```

C. Architecture Diagram (Text)

- USER INPUT
- Tender files (PDF/Excel/Word)
 - [Optional] Ingestion request (SAM/CanadaBuys)

INGESTION LAYER (Optional)

SAM.gov API	CanadaBuys
	CKAN API

api_metadata + attachments

DOCUMENT PROCESSING

Parser → Classifier → Section Extractor
 → Field Extractor → Keywords

TenderProfile
(api_metadata + doc_extracted)

VENDOR PIPELINE

Discovery → Enrichment → Filtering

LLM Capability Matching
(GPT-4o-mini → GPT-4o cascade)

OUTPUT GENERATION

XLSX / CSV / JSON → ./outputs/

Prepared by: Daria Pavlova **Report Version:** 1.0
Next Review: After Milestone 2